

Real-Time Tamil Fingerspelling Assistant

Executive summary

Real-Time Tamil Fingerspelling Assistant is a browser-based, bi-directional accessibility prototype grounded in the TLFS23 Tamil fingerspelling dataset and its motivation to support vision-based communication tools. It recognizes stable Tamil fingerspelling poses from a webcam and converts them into Tamil Unicode text. In the reverse direction, a user can enter Tamil text and view the corresponding reference sign image.

The project combines OpenCV 5 preprocessing, pretrained hand-pose estimation, deterministic landmark geometry, a Tamil fingerspelling mapping, temporal stability logic, and an AWS container deployment. The system is designed as a human-in-the-loop assistant: users see intermediate finger feedback and can correct their hand position before a sign is committed.

Live demo: [Real-Time Tamil Fingerspelling Assistant](#)

Source code: github.com/Dharun235/tamil-fingerspelling-assistant

Motivation and inspiration

Tamil fingerspelling is a useful bridge between hand pose and written language, but building a practical computer-vision assistant requires more than recognizing isolated gestures. The TLFS23 paper describes Tamil as a 247-character writing system and identifies the shortage of fingerspelling resources for vernacular languages as a barrier to assistive translator development. It presents TLFS23 as a labelled resource for vision-based fingerspelling research and communication support. See the [TLFS23 dataset article](#) and the [original dataset record](#).

This project turns that research resource into an interactive prototype. The goal is not to claim continuous sign-language translation. The goal is to give a person immediate, visible feedback while producing individual Tamil fingerspelling characters, then provide a visual reference when the direction is reversed.

Problem and users

Tamil has 247 characters represented in TLFS23, including vowels, consonants, combined characters, and the special Ayudha character. The paper describes the need for language-specific fingerspelling resources because systems built around more widely studied languages do not automatically provide a useful Tamil interface. A person producing a sign also needs more than a final prediction: they need to know whether the hand is visible, whether the fingers were interpreted correctly, and when the system has committed the character.

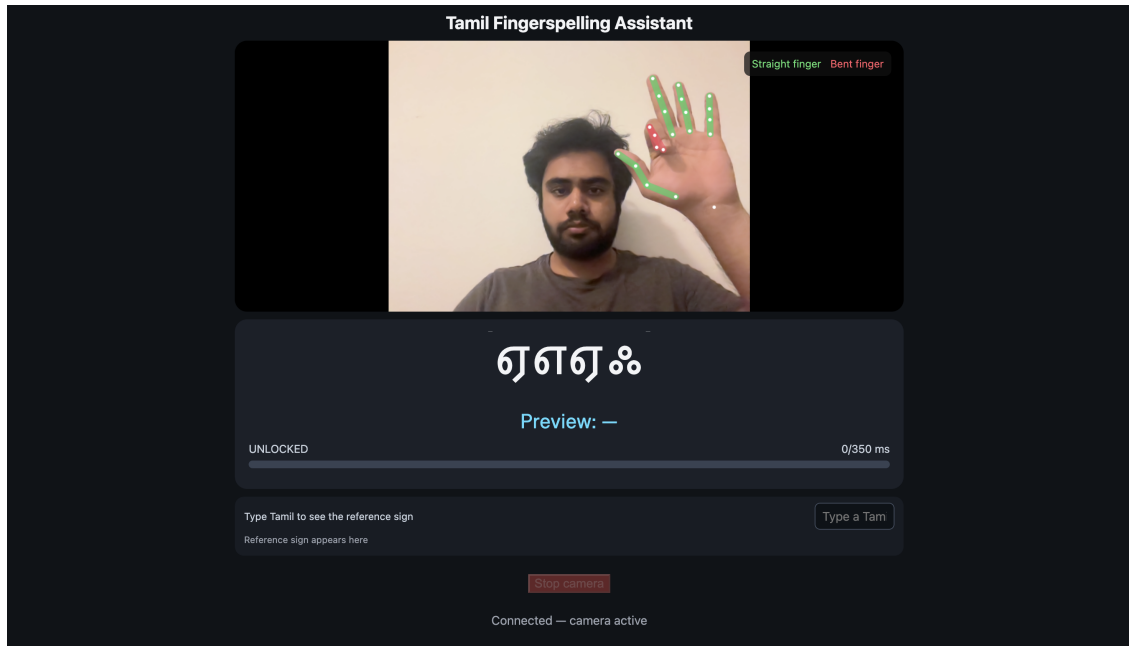
Intended users include people learning or using Tamil fingerspelling, communication partners, accessibility researchers, and developers building language interfaces. The reverse-reference panel also supports practice: a user can type a Tamil character and see the corresponding TLFS23 reference pose. This is an assistive prototype, not a certified medical, educational, or language-translation device.

What the system does

The application provides two connected workflows:

1. **Fingerspelling to text:** the browser captures webcam frames, the server detects up to two hands, and the application maps stable finger configurations to Tamil characters.
2. **Text to visual reference:** typed Tamil characters are matched to TLFS23 reference images so users can see the corresponding hand pose.

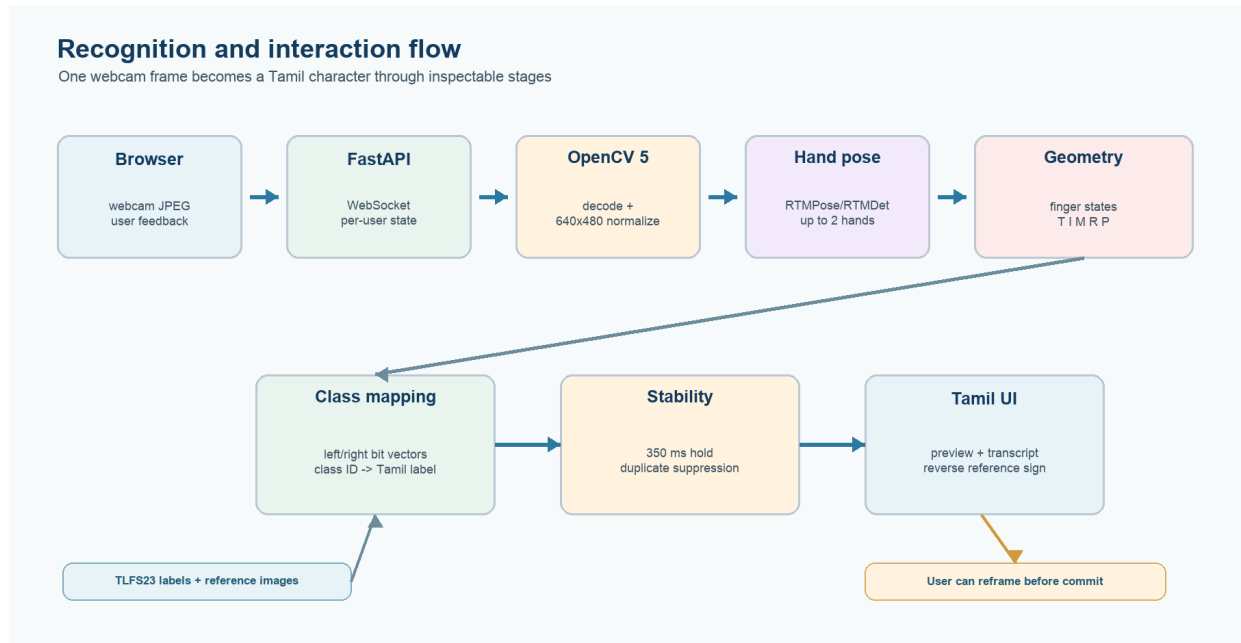
The interface shows the current pose, green/red finger-state feedback, the predicted character, the committed Tamil message, and a stability progress indicator. A sign must remain stable for approximately 350 ms before commitment. A fist inserts a space. The user can stop and restart camera capture without stopping the hosted service.



Live interface screenshot

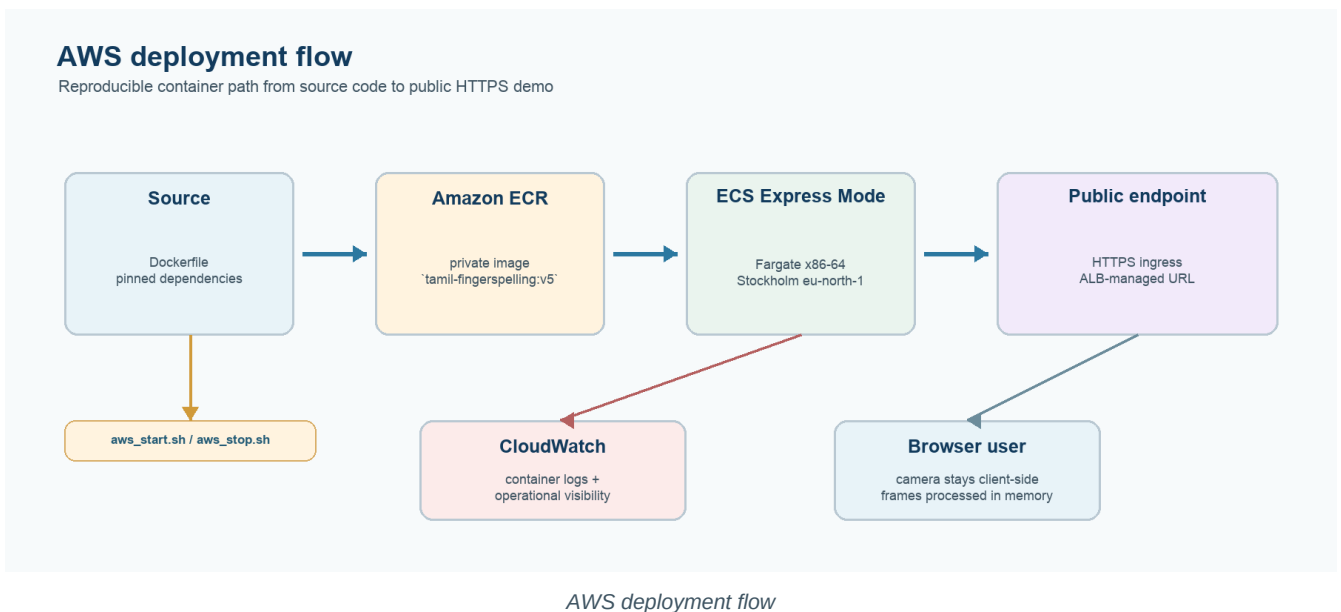
Figure 1. Live interface during recognition: webcam frame, hand-landmark overlay, Tamil output, stability timer, reverse-reference input, and connection status.

Architecture



Recognition and interaction flow

Figure 2. Recognition and interaction flow. Each stage produces inspectable intermediate output before the user-facing Tamil result.



AWS deployment flow

Figure 3. AWS deployment flow. The public demo is a containerized, reproducible path from source code to HTTPS endpoint.

The browser keeps camera capture client-side. Frames are processed in memory and are not intentionally persisted. The service is packaged as a Docker image and stored in private Amazon ECR. Amazon ECS Express Mode runs the image on Linux x86-64 AWS Fargate in eu-north-1 (Stockholm), with a public HTTPS ingress endpoint, autoscaling, and CloudWatch logging.

The repository includes the source diagrams in [docs/architecture.mmd](#), the rendered figures in [docs/assets/](#), local Docker configuration in [compose.yaml](#), and AWS lifecycle scripts in [aws_start.sh](#) and [aws_stop.sh](#).

OpenCV 5 implementation

OpenCV 5 is part of the runtime image path, not only a development dependency. For every incoming frame, the server:

1. decodes the browser JPEG into a BGR image;
2. validates the frame and normalizes it to the pipeline's fixed working size;
3. passes the normalized image to the hand-pose pipeline;
4. uses the resulting landmarks for downstream geometry and mapping.

The project pins `opencv-python==5.0.0.93` and checks the OpenCV major version during application startup. The OpenCV operation is meaningful to the application because every recognition result depends on successful frame decoding and normalization before hand analysis.

The remaining vision pipeline uses RTMLib with pretrained RTMPose/RTMDet models and ONNX Runtime CPU inference. This is an inference-and-integration project; it does not train a project-specific neural network.

Recognition and interaction design

The detector produces hand landmarks. Geometry functions normalize distances and relative positions so finger rules are less dependent on image scale. Each finger is classified as open or closed, and the resulting state is mapped to a Tamil class.

The temporal state machine prevents a held sign from producing repeated characters. It also exposes uncertainty rather than silently committing every frame: users can see red/green finger feedback, move their hands into frame, and hold the corrected pose until the progress indicator completes.

This design deliberately separates three stages of evidence:

- hand detection and side assignment;
- finger-state inference;
- exact Tamil class mapping and temporal commitment.

That separation makes failures inspectable and avoids presenting a final character score as if it were a pure hand-pose accuracy measurement.

Dataset class mapping

TLFS23 supplies the class vocabulary. Its `ReadMe.txt` maps numeric class IDs to Tamil Unicode labels, while the reference-image directory contains one representative sign image for each character. The application loads this label table at startup; it does not infer Tamil Unicode from the pixels.

The recognition mapping is deterministic and uses five fingers in this order:

T I M R P

Each hand becomes a five-bit open/closed vector. For example, `01000` means index finger open, while `11000` means thumb and index open. The mapping has three ranges: classes 1–13 are right-hand-only patterns, classes

14–31 are left-hand-only patterns, and classes 32–247 combine 18 left-hand patterns with 12 right-hand patterns to represent 216 Tamil characters. The background class is excluded from Tamil output.

Representative entries are shown in the reference-sign figure below; the full table appears in the appendix.

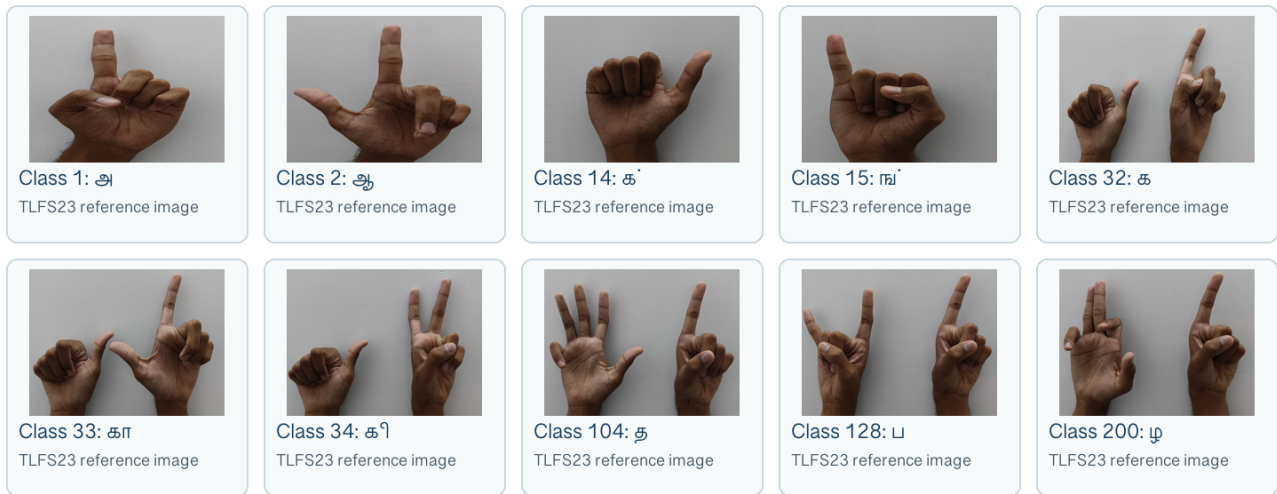
The runtime flow is therefore:

landmarks -> finger states -> left/right bit vectors -> class ID -> Tamil Unicode label

Examples below show the actual TLFS23 reference signs used by the reverse-reference panel. They also make the distinction clear between a visual sign, its numeric class ID, and its Tamil label.

TLFS23 class IDs -> Tamil labels -> reference signs

Examples from the 247-character mapping; bit order: T I M R P (1 = finger open)



TLFS23 class-to-sign mapping examples

Figure 4. Representative TLFS23 mappings. Bit order is T-I-M-R-P; 1 means open.

The complete 247-class table is available in [docs/TLFS23_CLASS_MAPPING.md](#) and is included as an appendix in the submission PDF.

Appendix - complete TLFS23 class mapping

Class ID	Tamil label	Left fingers	Left bits	Right fingers	Right bits
1	அ	-	-	I	01000
2	ஆ	-	-	T, I	11000
3	இ	-	-	I, M	01100
4	ஈ	-	-	T, I, M	11100
5	உ	-	-	I, M, R	01110
6	ஊ	-	-	T, I, M, R	11110
7	எ	-	-	I, M, R, P	01111
8	ஏ	-	-	T, I, M, R, P	11111
9	ஐ	-	-	T	10000
10	ஓ	-	-	P	00001
11	ஔ	-	-	T, P	10001
12	ஓள	-	-	I, P	01001
13	ஊ	-	-	M, R, P	00111
14	க'	T	10000	-	-
15	ங'	P	00001	-	-
16	ச'	P, R, M	00111	-	-
17	ஞ'	P, R	00011	-	-
18	ட'	T, I	11000	-	-
19	ண'	I, M, R	01110	-	-
20	த'	T, I, M, R	11110	-	-
21	ந'	I	01000	-	-
22	ப'	I, P	01001	-	-
23	ம'	T, I, M, P	11101	-	-
24	ய'	I, M, R, P	01111	-	-
25	ர'	T, M	10100	-	-
26	ல'	T, P	10001	-	-

Class ID	Tamil label	Left fingers	Left bits	Right fingers	Right bits
27	வ்	T, M, R, P	10111	-	-
28	ழ்	T, M, R	10110	-	-
29	ள்	T, I, P	11001	-	-
30	ற்	T, I, M	11100	-	-
31	ன்	I, M	01100	-	-
32	க	T	10000	I	01000
33	கா	T	10000	T, I	11000
34	கி	T	10000	I, M	01100
35	கீ	T	10000	T, I, M	11100
36	கு	T	10000	I, M, R	01110
37	கூ	T	10000	T, I, M, R	11110
38	கெ	T	10000	I, M, R, P	01111
39	கே	T	10000	T, I, M, R, P	11111
40	கை	T	10000	T	10000
41	கொ	T	10000	P	00001
42	கோ	T	10000	T, P	10001
43	கௌ	T	10000	I, P	01001
44	ங	P	00001	I	01000
45	ஙா	P	00001	T, I	11000
46	ஙி	P	00001	I, M	01100
47	ஙீ	P	00001	T, I, M	11100
48	ஙு	P	00001	I, M, R	01110
49	ஙூ	P	00001	T, I, M, R	11110
50	ஙெ	P	00001	I, M, R, P	01111
51	ஙே	P	00001	T, I, M, R, P	11111
52	ஙை	P	00001	T	10000
53	ஙொ	P	00001	P	00001

Class ID	Tamil label	Left fingers	Left bits	Right fingers	Right bits
54	ஙோ	P	00001	T, P	10001
55	ஙொ	P	00001	I, P	01001
56	ச	P, R, M	00111	I	01000
57	சா	P, R, M	00111	T, I	11000
58	சௌ	P, R, M	00111	I, M	01100
59	சே	P, R, M	00111	T, I, M	11100
60	சு	P, R, M	00111	I, M, R	01110
61	சு	P, R, M	00111	T, I, M, R	11110
62	செ	P, R, M	00111	I, M, R, P	01111
63	சே	P, R, M	00111	T, I, M, R, P	11111
64	சை	P, R, M	00111	T	10000
65	சொ	P, R, M	00111	P	00001
66	சோ	P, R, M	00111	T, P	10001
67	சொ	P, R, M	00111	I, P	01001
68	ரு	P, R	00011	I	01000
69	ருா	P, R	00011	T, I	11000
70	ருௌ	P, R	00011	I, M	01100
71	ருே	P, R	00011	T, I, M	11100
72	ரு	P, R	00011	I, M, R	01110
73	ரு	P, R	00011	T, I, M, R	11110
74	ருெ	P, R	00011	I, M, R, P	01111
75	ருே	P, R	00011	T, I, M, R, P	11111
76	ருை	P, R	00011	T	10000
77	ருொ	P, R	00011	P	00001
78	ருோ	P, R	00011	T, P	10001
79	ருொ	P, R	00011	I, P	01001
80	ல	T, I	11000	I	01000

Class ID	Tamil label	Left fingers	Left bits	Right fingers	Right bits
81	டா	T, I	11000	T, I	11000
82	டி	T, I	11000	I, M	01100
83	டீ	T, I	11000	T, I, M	11100
84	டூ	T, I	11000	I, M, R	01110
85	டூ	T, I	11000	T, I, M, R	11110
86	டெ	T, I	11000	I, M, R, P	01111
87	டே	T, I	11000	T, I, M, R, P	11111
88	டை	T, I	11000	T	10000
89	டொ	T, I	11000	P	00001
90	டோ	T, I	11000	T, P	10001
91	டொ	T, I	11000	I, P	01001
92	ண	I, M, R	01110	I	01000
93	ணா	I, M, R	01110	T, I	11000
94	ணி	I, M, R	01110	I, M	01100
95	ணீ	I, M, R	01110	T, I, M	11100
96	ணூ	I, M, R	01110	I, M, R	01110
97	ணூ	I, M, R	01110	T, I, M, R	11110
98	ணெ	I, M, R	01110	I, M, R, P	01111
99	ணே	I, M, R	01110	T, I, M, R, P	11111
100	ணை	I, M, R	01110	T	10000
101	ணொ	I, M, R	01110	P	00001
102	ணோ	I, M, R	01110	T, P	10001
103	ணொ	I, M, R	01110	I, P	01001
104	த	T, I, M, R	11110	I	01000
105	தா	T, I, M, R	11110	T, I	11000
106	தி	T, I, M, R	11110	I, M	01100
107	தீ	T, I, M, R	11110	T, I, M	11100

Class ID	Tamil label	Left fingers	Left bits	Right fingers	Right bits
108	த஁	T, I, M, R	11110	I, M, R	01110
109	தஃ	T, I, M, R	11110	T, I, M, R	11110
110	த஄	T, I, M, R	11110	I, M, R, P	01111
111	தஅ	T, I, M, R	11110	T, I, M, R, P	11111
112	தஆ	T, I, M, R	11110	T	10000
113	தஇ	T, I, M, R	11110	P	00001
114	தஈ	T, I, M, R	11110	T, P	10001
115	தஉ	T, I, M, R	11110	I, P	01001
116	ந	I	01000	I	01000
117	நா	I	01000	T, I	11000
118	நஃ	I	01000	I, M	01100
119	ந஄	I	01000	T, I, M	11100
120	நஅ	I	01000	I, M, R	01110
121	நஆ	I	01000	T, I, M, R	11110
122	நஇ	I	01000	I, M, R, P	01111
123	நஈ	I	01000	T, I, M, R, P	11111
124	நஉ	I	01000	T	10000
125	நஊ	I	01000	P	00001
126	ந஋	I	01000	T, P	10001
127	ந஌	I	01000	I, P	01001
128	ப	I, P	01001	I	01000
129	பா	I, P	01001	T, I	11000
130	பஃ	I, P	01001	I, M	01100
131	ப஄	I, P	01001	T, I, M	11100
132	பஅ	I, P	01001	I, M, R	01110
133	பஆ	I, P	01001	T, I, M, R	11110
134	பஇ	I, P	01001	I, M, R, P	01111

Class ID	Tamil label	Left fingers	Left bits	Right fingers	Right bits
135	பு	I, P	01001	T, I, M, R, P	11111
136	பு	I, P	01001	T	10000
137	பு	I, P	01001	P	00001
138	பு	I, P	01001	T, P	10001
139	பு	I, P	01001	I, P	01001
140	ம	T, I, M, P	11101	I	01000
141	மா	T, I, M, P	11101	T, I	11000
142	ம	T, I, M, P	11101	I, M	01100
143	ம	T, I, M, P	11101	T, I, M	11100
144	ம	T, I, M, P	11101	I, M, R	01110
145	ம	T, I, M, P	11101	T, I, M, R	11110
146	ம	T, I, M, P	11101	I, M, R, P	01111
147	ம	T, I, M, P	11101	T, I, M, R, P	11111
148	ம	T, I, M, P	11101	T	10000
149	ம	T, I, M, P	11101	P	00001
150	ம	T, I, M, P	11101	T, P	10001
151	ம	T, I, M, P	11101	I, P	01001
152	ய	I, M, R, P	01111	I	01000
153	யா	I, M, R, P	01111	T, I	11000
154	ய	I, M, R, P	01111	I, M	01100
155	ய	I, M, R, P	01111	T, I, M	11100
156	ய	I, M, R, P	01111	I, M, R	01110
157	ய	I, M, R, P	01111	T, I, M, R	11110
158	ய	I, M, R, P	01111	I, M, R, P	01111
159	ய	I, M, R, P	01111	T, I, M, R, P	11111
160	ய	I, M, R, P	01111	T	10000
161	ய	I, M, R, P	01111	P	00001

Class ID	Tamil label	Left fingers	Left bits	Right fingers	Right bits
162	யோ	I, M, R, P	01111	T, P	10001
163	யெ்ள	I, M, R, P	01111	I, P	01001
164	ர	T, M	10100	I	01000
165	ரா	T, M	10100	T, I	11000
166	ரீ	T, M	10100	I, M	01100
167	ரீ	T, M	10100	T, I, M	11100
168	ரூ	T, M	10100	I, M, R	01110
169	ரூ	T, M	10100	T, I, M, R	11110
170	ரெ	T, M	10100	I, M, R, P	01111
171	ரே	T, M	10100	T, I, M, R, P	11111
172	ரண	T, M	10100	T	10000
173	ரெொ	T, M	10100	P	00001
174	ரோ	T, M	10100	T, P	10001
175	ரெ்ள	T, M	10100	I, P	01001
176	ல	T, P	10001	I	01000
177	லா	T, P	10001	T, I	11000
178	லீ	T, P	10001	I, M	01100
179	லீ	T, P	10001	T, I, M	11100
180	லூ	T, P	10001	I, M, R	01110
181	லூ	T, P	10001	T, I, M, R	11110
182	லெ	T, P	10001	I, M, R, P	01111
183	லே	T, P	10001	T, I, M, R, P	11111
184	லண	T, P	10001	T	10000
185	லெொ	T, P	10001	P	00001
186	லோ	T, P	10001	T, P	10001
187	லெ்ள	T, P	10001	I, P	01001
188	வ	T, M, R, P	10111	I	01000

Class ID	Tamil label	Left fingers	Left bits	Right fingers	Right bits
189	வா	T, M, R, P	10111	T, I	11000
190	வாி	T, M, R, P	10111	I, M	01100
191	வாீ	T, M, R, P	10111	T, I, M	11100
192	வாு	T, M, R, P	10111	I, M, R	01110
193	வாு	T, M, R, P	10111	T, I, M, R	11110
194	வாெ	T, M, R, P	10111	I, M, R, P	01111
195	வாே	T, M, R, P	10111	T, I, M, R, P	11111
196	வாை	T, M, R, P	10111	T	10000
197	வாொ	T, M, R, P	10111	P	00001
198	வாோ	T, M, R, P	10111	T, P	10001
199	வாொள	T, M, R, P	10111	I, P	01001
200	ழ	T, M, R	10110	I	01000
201	ழா	T, M, R	10110	T, I	11000
202	ழாி	T, M, R	10110	I, M	01100
203	ழாீ	T, M, R	10110	T, I, M	11100
204	ழாு	T, M, R	10110	I, M, R	01110
205	ழாு	T, M, R	10110	T, I, M, R	11110
206	ழாெ	T, M, R	10110	I, M, R, P	01111
207	ழாே	T, M, R	10110	T, I, M, R, P	11111
208	ழாை	T, M, R	10110	T	10000
209	ழாொ	T, M, R	10110	P	00001
210	ழாோ	T, M, R	10110	T, P	10001
211	ழாொள	T, M, R	10110	I, P	01001
212	ள	T, I, P	11001	I	01000
213	ளா	T, I, P	11001	T, I	11000
214	ளாி	T, I, P	11001	I, M	01100
215	ளாீ	T, I, P	11001	T, I, M	11100

Class ID	Tamil label	Left fingers	Left bits	Right fingers	Right bits
216	எஃ	T, I, P	11001	I, M, R	01110
217	எஃ	T, I, P	11001	T, I, M, R	11110
218	எஃ	T, I, P	11001	I, M, R, P	01111
219	எஃ	T, I, P	11001	T, I, M, R, P	11111
220	எஃ	T, I, P	11001	T	10000
221	எஃ	T, I, P	11001	P	00001
222	எஃ	T, I, P	11001	T, P	10001
223	எஃ	T, I, P	11001	I, P	01001
224	ற	T, I, M	11100	I	01000
225	றா	T, I, M	11100	T, I	11000
226	றா	T, I, M	11100	I, M	01100
227	றா	T, I, M	11100	T, I, M	11100
228	றா	T, I, M	11100	I, M, R	01110
229	றா	T, I, M	11100	T, I, M, R	11110
230	றா	T, I, M	11100	I, M, R, P	01111
231	றா	T, I, M	11100	T, I, M, R, P	11111
232	றா	T, I, M	11100	T	10000
233	றா	T, I, M	11100	P	00001
234	றா	T, I, M	11100	T, P	10001
235	றா	T, I, M	11100	I, P	01001
236	ன	I, M	01100	I	01000
237	னா	I, M	01100	T, I	11000
238	னா	I, M	01100	I, M	01100
239	னா	I, M	01100	T, I, M	11100
240	னா	I, M	01100	I, M, R	01110
241	னா	I, M	01100	T, I, M, R	11110
242	னா	I, M	01100	I, M, R, P	01111

Class ID	Tamil label	Left fingers	Left bits	Right fingers	Right bits
243	மாடு	I, M	01100	T, I, M, R, P	11111
244	மாலை	I, M	01100	T	10000
245	மாலை	I, M	01100	P	00001
246	மாலை	I, M	01100	T, P	10001
247	மாலை	I, M	01100	I, P	01001

Source: TLFS23 on Mendeley Data, CC BY 4.0.

Dataset and attribution

The project uses **TLFS23 — Tamil Language Finger Spelling Image Dataset** for Tamil character labels, reverse-direction reference images, and offline evaluation. TLFS23 contains 248 classes: 247 Tamil characters plus background, with approximately 255,155 images collected from 120 volunteers.

Source and attribution: Chirranjeavi et al., “TLFS23 - Tamil Language Finger Spelling Image Dataset,” Mendeley Data, version 2, 2023, DOI [10.17632/39kzs5pxmk.2](https://doi.org/10.17632/39kzs5pxmk.2), licensed under [CC BY 4.0](#). The related [dataset article](#) describes its collection and intended research use.

The dataset is not committed to this repository. Users must obtain it from the original source and comply with its license and attribution requirements. The camera-recognition path can run without reference images; the reverse visual-reference panel requires the Reference Image assets. The evaluator uses the class folders as proxy labels and does not claim manually verified landmark ground truth.

Evaluation

Static dataset evaluation

The reproducible benchmark samples 20 images from each of the 247 non-background classes: 4,940 images total. It writes one row per image plus aggregate and per-class outputs under `results/dataset_eval_20/`.

Stage	Result
Expected left hands detected	84.5% (3,956/4,680)
Expected right hands detected	83.5% (3,826/4,580)
Exact detected hand-side set	70.7%
Finger-state accuracy	92.5–97.3% per finger
Exact folder-class match	54.8%
Mean image throughput	8.6 images/s

These results must be interpreted carefully. Folder labels are proxy labels and are not manually verified landmark ground truth. The photographed dataset includes occlusion, merged hands, poor framing, and out-of-frame examples. The largest bottleneck is hand detection and landmark quality. The 54.8% exact folder-class result should not be presented as real-time user accuracy.

Latency by environment

Latency is reported in separate layers because each measurement answers a different question. The local benchmark ran on this Mac over 4,940 TLFS23 images. The AWS measurement sent 10 warm JPEG frames through the live WebSocket endpoint and included network transfer plus server inference. Neither measurement includes webcam capture, browser rendering, or the stability hold.

Measurement	Mean	P50	P95	Includes
Mac local offline inference	116.6 ms	109.9 ms	193.3 ms	OpenCV decode, resize, hand pose, geometry, mapping
AWS warm WebSocket request	113.8 ms	113.4 ms	115.6 ms	JPEG request/response transfer + AWS CPU inference
User-visible character commit	Not measured	Not measured	Not measured	Webcam, browser, network, inference, rendering, plus 350 ms stable hold

The Mac run also measured a p99 of 248.8 ms. The AWS result is a small warm-sample measurement, not a capacity or long-term service guarantee. A proper end-to-end live test should record repeated signs and report commit delay separately from server latency.

Real-time evaluation

Live interaction answers a different question because the user receives feedback and can correct framing. The recommended live protocol uses representative one-hand, two-hand, overlap, fist/space, and difficult-framing signs, repeated three times each. Record committed-character accuracy, missed commits, duplicate commits, median/p95 commit delay, and visible behavior.

End-to-end delay includes webcam capture, JPEG encoding, network transfer, WebSocket scheduling, CPU inference, rendering, and the 350 ms stability requirement. It should therefore be reported separately from both the Mac offline result and the AWS warm WebSocket result above.

Limitations

- Occlusion, overlapping hands, lighting, camera angle, and out-of-frame poses can reduce detection quality.
- Geometry rules are sensitive to landmark errors and unusual hand orientations.
- The static dataset benchmark does not represent the user-correction loop in the live interface.
- The application recognizes individual fingerspelling characters; it is not continuous Tamil sign-language translation.
- The hosted service uses CPU inference, so latency varies with Fargate load and network conditions.
- The reverse-direction reference images require the separately obtained TLFS23 assets.
- The prototype has not been evaluated as a clinical, educational, or safety-critical system.

Responsible use and privacy

The application should be described as an assistive prototype. Users should not rely on it for medical, legal, emergency, or safety-critical communication without independent confirmation. Recognition output can be wrong,

especially under occlusion or poor framing, so the interface exposes intermediate feedback and allows the user to repeat a sign.

Camera frames are sent to the application for inference and are not intentionally written to disk by this project. Operators should still apply appropriate AWS access controls, logging policies, retention settings, and privacy notices when running a public deployment. Anyone collecting new images should obtain informed consent, minimize personally identifying data, and publish clear data-use and deletion policies.

Reproducibility

The repository provides:

- pinned Python dependencies in [requirements.txt](#);
- local Python and Docker instructions in [README.md](#);
- a reproducible evaluator in [scripts/evaluate_dataset.py](#);
- architecture documentation and Mermaid source;
- AWS start/stop scripts;
- a warm live-endpoint latency probe in [scripts/measure_live_latency.py](#);
- third-party notices and dataset attribution.

The live project endpoint is:

<https://ta-89507d6158f8458c88ac7b38194d8efc.ecs.eu-north-1.on.aws>

Demo presentation content

For a short judge presentation, use this sequence:

1. **Need:** Tamil fingerspelling users need immediate confirmation and a visual reference for signs.
2. **Live recognition:** show the webcam, green/red finger feedback, stability bar, and Tamil output.
3. **Bi-directional workflow:** type a Tamil character and show its TLFS23 reference pose.
4. **Technical path:** browser → WebSocket → OpenCV 5 → hand landmarks → geometry rules → Tamil mapping.
5. **Cloud delivery:** Docker → ECR → ECS Express Mode/Fargate → HTTPS endpoint → CloudWatch.
6. **Evidence and limits:** show the benchmark results, then explain occlusion, proxy labels, and human-in-the-loop correction.

The key message is: **a browser-based assistant turns Tamil fingerspelling into visible text while helping users correct their pose in real time, and the same reproducible pipeline runs on AWS.**

Conclusion

This project demonstrates a practical path from webcam input to Tamil fingerspelling assistance using OpenCV 5 and AWS. Its main contribution is the complete, inspectable workflow: vision preprocessing, hand landmarks, geometric interpretation, temporal interaction, reverse visual references, evaluation evidence, and a deployable cloud endpoint. Future work will improve robustness across users and environments, add larger real-world live

evaluations, and explore speech output and broader Tamil sign support.